



Minimal SQL Agent Demo (Technical Exposition)

Completion requirements

DSC 360 – Minimal SQL Agent Demo (Technical Exposition)

Quick Start (2 min)

1. Export your OpenAI key in this shell:
`export OPENAI_API_KEY='sk-...'`
2. Set MySQL env (use your Lab 6 creds; DB is **gravity_books** on **cscdata.centre.edu**):
`export DB_HOST='cscdata.centre.edu'; export DB_NAME='gravity_books'`
`export DB_USER='team_user'; export DB_PASSWORD='.....'`
3. Run: `python3 agent_sql_demo_min.py`

1) Overview (what it is, why we built it)

This demo is a **small, dependable, agentic loop** that converts a natural-language goal into a safe SQL query, runs the query against a **read-only MySQL database**, inspects a small preview of results, and decides whether the goal was met or if another attempt is needed.

- **Three roles (sequential, single-agent loop):**
 1. **Planner** – reads the goal + schema hint and proposes a short plan (and maybe a first query).
 2. **SQL Writer** – proposes exactly one **read-only** SQL statement that should move us toward the goal.
 3. **Verifier** – inspects a preview of results and says **accept / revise / stop** with a short rationale.
- **Why this shape?** It's simple enough to run live, but it still demonstrates **agentic iteration** (think: “reason → act → observe → adjust”), aligns with what we've covered (structured extraction, safe NL→SQL), and avoids the pitfalls of long, fragile pipelines.

2) What kind of agent is this? (R&N framing)

Russell & Norvig taxonomy:

- **Type: A goal-based, model-based problem-solving agent.**
 - *Goal-based*: every run starts from a user goal (“List top languages...”) and stops when the Verifier accepts or we hit an iteration cap.

- *Model-based*: it maintains a small internal model of the world: a **schema hint** (introspected from `information_schema`) + a record of the last outcome (“empty sample”, “blocked by validator”, etc.).
- *Problem-solving*: each iteration is a micro-search over SQLs; we don’t learn over time, and we don’t optimize a utility over long horizons.
- **PEAS specification**:
 - **Performance**: correctness of results; **read-only safety**; latency (<~2–10s per turn); single-statement discipline; clear rationale.
 - **Environment**: MySQL DB (read-only session), OpenAI LLM, user REPL, OS/network; schema discovered at runtime.
 - **Actuators**: (a) send prompts to LLM; (b) execute **one** SQL statement on MySQL.
 - **Sensors**: (a) LLM JSON outputs (plan/draft/verdict); (b) **schema hint** built from `information_schema`; (c) **preview rows** and SQL errors; (d) our validator’s accept/block messages.
- **Knowledge base (KB)**:
 - **Structured prompts** (system instructions for Planner/Writer/Verifier).
 - **Live schema hint** distilled from `information_schema` (one compact line per table).
 - **Read-only rules** (single-statement, allowed verbs, blocklist).
 - **Runtime state**: last proposed SQL, last error (“blocked by validator...”, “execution error...”) or result preview.

3) How it works (the loop at a glance)

REPL flow: user types a goal → agent runs (**Plan** → **Write** → **Execute** → **Verify**) → either **accept** or loop another try (bounded, e.g., 2–3 iterations).

Pseudocode (sketch):

```
goal ← user input
schema ← build_schema_hint(conn)  ### from information_schema

plan ← LLM(Planner, goal + schema)

sql ← plan.initial_sql
prior_issue ← ""

repeat up to MAX_ITERS:
    draft ← LLM(SQL Writer, goal + schema + steps + sql + prior_issue)
    sql ← normalize(sql from draft)
    if !read_only_and_single_statement(sql): prior_issue ← "blocked"; continue
    (cols, rows, total) ← execute_preview(sql, buffered=True, max_rows=10)
```

```

    if rows is empty and goal likely implies rows:
        prior_issue ← "sample empty; check joins/filters or schema support";
        continue

    verdict ← LLM(Verifier, goal + schema + sql + sample(rows))
    if verdict ∈ {accept, stop}: return final
    prior_issue ← verdict.reason/issues
return "stop: max iterations"

```

Note: If the goal implies rows and preview is empty, the verifier asks for a revision (typical join/filter issue)."

4) Key engineering choices (and why they matter)

Safety envelope (lite but robust for demo)

- **Read-only validator:** only allows `WITH / SELECT / SHOW / DESCRIBE`, one statement, blocks DDL/DML (`INSERT/UPDATE/DELETE/ALTER/...`).
- **Session hardening:** `SET SESSION TRANSACTION READ ONLY` (best-effort) + short **execution timeout** (`MAX_EXECUTION_TIME=2000ms`) to prevent stallers.
- **Optional pre-flight refusal:** If the **goal** contains destructive verbs ("delete all titles..."), we **stop immediately** with a friendly refusal. This avoids tedious "blocked...try again" loops.

Reliability and UX

- **Buffered cursor** on MySQL preview to consume the result set completely → fixes the classic `Unread result found` error on subsequent queries.
- **Live schema introspection** (compact hint from `information_schema`) → avoids guessing table names; strongly reduces join-name errors.
- **Small previews** (10 rows) with a **simple ASCII table** → makes verification human-legible and keeps latency low.
- **Low temperature & bounded iterations** → less variance live; "good enough" in 1–3 tries; predictable demos.
- **Pydantic v2 for structured JSON** → stable parsing for plan/draft/verdict; graceful fallback if the model returns text.

Prompt rules (general, not hard-coded cases)

- **Writer best-practices:**
 - Prefer `NOT EXISTS` (or `LEFT JOIN ... IS NULL`) over `NOT IN` to dodge `NULL` traps.
 - Use `COALESCE(SUM(...), 0)` when aggregating with left joins.
 - For **top-N**, require `ORDER BY` and `LIMIT`.
 - **Group by natural keys** when aggregating across variants (e.g., **title**, not `book_id`, when counting languages across editions).

- **Verifier discipline:**
 - Don't accept empty samples when the goal implies rows (revise or stop).
 - Ensure requested columns/aliases are present and **non-NULL** in sample rows.
 - Require `ORDER BY+LIMIT` when **top-N** is implied.
 - Flag "`COUNT(DISTINCT X)` but `GROUP BY` includes a unique row ID" as suspicious.

These prompts are **small, durable nudges** that improved behavior without "overtraining" on one schema.

5) Hurdles we hit (and how we resolved them)

1. **Context bloat / brittle regex chains**
 - Early versions tried to micro-patch every failure case; the loop got fragile.
 - **Fix:** shrink roles, use **live schema**, keep **short prompts** with general best-practices, and bound iterations.
2. **"Unread result found" (MySQL)**
 - Came from using an unbuffered cursor + only `fetchmany`.
 - **Fix: buffered dictionary cursor**, always consumes the result set; close cursor each time.
3. **Wrong table names / joins**
 - The model hallucinated names (`order_item` vs `order_line`).
 - **Fix: introspect schema** at runtime and pass that hint to the Planner/Writer.
4. **Verifier too lenient**
 - Accepted empty results or nulls when the task clearly required data.
 - **Fix:** prompt rules (non-empty when implied, required columns must be present & non-NULL, top-N needs ordering).
5. **"Delete/censor" red-team prompt**
 - Writer kept trying `DELETE`, then got blocked.
 - **Fix (optional, minimal):** pre-flight refusal if the **goal text** requests mutation; remind Writer it's **read-only**.
6. **The "translated into >2 languages" trap**
 - Grouping by `book_id` capped language count at 1; empty sample.
 - **Fix:** Writer/Verifier nudges to group by **title** (natural key) for that report.
 - **Correct query you confirmed in MySQL:**

```
sql SELECT b.title, COUNT(DISTINCT b.language_id) AS language_count FROM
book AS b GROUP BY b.title HAVING COUNT(DISTINCT b.language_id) > 2 ORDER
BY language_count DESC, b.title;
```
7. **ASCII in env vars**

- Non-ASCII paste in `OPENAI_API_KEY` triggers `UnicodeEncodeError` deep in http headers.
 - **Fix:** quick ASCII check up-front with a clear error message.
-

6) How this connects to earlier course topics

- **Chatbot prompting** → we formalized roles (Planner/Writer/Verifier) and gave each a focused instruction set.
- **Semantic search / RAG** → schema introspection is RAG-like: **retrieve** the schema from the DB and **condition** the LLM.
- **Structured extraction** → role outputs are **Pydantic-validated JSON**, not free-form text.
- **Safe NL→SQL** → we reuse the **read-only validator**, and the Verifier does a light **post-hoc check** on outputs.
- **Agents** → instead of a one-shot pipeline, the agent **iterates**: observe → propose → act → observe → adjust, with a **bounded** loop and simple state.

This demo shows how basic agentic structure + good guardrails can **substantially raise reliability** without heavy frameworks.

7) Important considerations for your own projects

- **Minimize moving parts first.** Small roles + short prompts + bounded loops beat big pipelines when you're live.
 - **Always shape the environment.** Get a **live KB** (schema, examples, constraints) instead of letting the model guess.
 - **Make safety cheap.** Denylist/allowlist + single-statement validation + read-only session is ~10 lines and prevents 90% of disasters.
 - **Instrument the loop.** Print plan/SQL/preview/verdict. Humans should be able to audit what happened in seconds.
 - **Prefer general prompt rules to case rules.** ("Group by natural key" beats "for **book** use **title**".)
 - **Keep latency under control.** Small previews, low temperature, short timeouts, few iterations.
-

8) What we'd do with more time (next steps)

- **Static SQL lint/parse pass** (e.g., syntax check; warn on cartesian joins; surface missing `ORDER BY` in "top-N").
- **Join-key guidance from foreign keys** (derive likely join paths from `information_schema.KEY_COLUMN_USAGE`).

- **Transparent, user-facing critique** (show why a query was revised; link issues to plan steps).
- **Test harness** (small library of NL goals ↔ gold SQL ↔ gold-result fingerprints).
- **Deeper safety** (table/column allow-lists, per-role prompts for sensitive topics, goal-screening).
- **Caching** (memoize schema + common subgoals to cut latency).

9) Informal evaluation (what we saw in our brief experiments)

- **Works well** on “never ordered” and “customer total spend” tasks.
- **Initially weak** on “translated into >2 languages” until we added general grouping guidance and stricter verification.
- **Red-team “delete/censor”** prompts were blocked by the validator; optional pre-flight refusal improves UX.

10) Conclusion

This is a **compact, readable, and safe** example of an **agentic loop** that integrates the course’s earlier ideas—retrieval (schema), structured output (Pydantic), safe execution (validator, read-only), and iteration (Planner/Writer/Verifier). It’s ready for demo, and it’s a practical template students can extend thoughtfully (e.g., add a linter, join-key discovery, or a richer Verifier) without making the system brittle.

Adapting to Ollama (gemma3:12b, Pydantic v2 schema)

We keep the same contract but validate with Pydantic locally: `ask_json(system, user, Model) -> dict`

```
import json, ollama

def ask_json(system_prompt: str, user_prompt: str, Model) -> dict:
    # Pydantic v2 JSON Schema (strict shape we expect back)
    schema = Model.model_json_schema()

    resp = ollama.chat(
        model='gemma3:12b',
        messages=[
            {'role': 'system',
             'content': "Return a SINGLE JSON object that matches this JSON
Schema:\n"
                        + json.dumps(schema)},
            {'role': 'user',
             'content': user_prompt}
        ],
```

```
        options={'temperature': 0},
        format='json' # ask Ollama to output valid JSON only
    )

    data = json.loads(resp['message']['content'])
    return Model.model_validate(data).model_dump() # enforce schema locally
```

Last modified: Thursday, November 6, 2025, 5:32 PM